Reporte del Proyecto GM

Videojuego Recolectar Gotas

Asignatura	INF2236 - Programación Avanzada
Profesor	Claudio Cubillos
Período	2do Semestre 2026
Integrantes	Nicolas Jaramillo, Lucas Ahumada, Alessandro Reginato
Fecha de entrega	25/10/2026

Resumen del proyecto

Juego base: GameLluviaMenu2024

Tecnologías utilizadas: Java 11, LibGDX 1.12.1, Gradle, GitHub y NetBeans

Forma de ejecución: aplicación de escritorio mediante el módulo lwjgl3

Tabla de Contenidos

Sección	Página
GM-1 Explicación del juego desde la perspectiva del jugador	2-3
GM-2 Análisis del juego desde la perspectiva de ingeniería de software	3-4
GM-3 Diseño de diagrama UML con clases del Dominio del juego	5
GM-4 Diseño y codificación de clase abstracta e interfaz	6
GM-5 Aplicación de encapsulamiento y principios de orientación a objetos	7
GM-6 Aplicación del patrón de diseño Singleton	8
GM-7 Aplicación del patrón de diseño Strategy	9
GM-8 Aplicación del patrón de diseño Template Method	10
GM-9 Aplicación del patrón de diseño Builder	11
GM-10 Utilización de GitHub	12

GM-1 Explicación del juego desde la perspectiva del jugador

Requisito solicitado: Explicación del juego desde la perspectiva del jugador, incluyendo objetivos, narrativa, elementos y controles, utilizando maquetas o pantallazos.

Objetivos

Nuestro juego consiste en recolectar gotas buenas y evitar las gotas dañinas utilizando un tarro. El objetivo es conseguir la mayor cantidad de puntos antes de perder todas las vidas.

El jugador puede elegir entre el modo fácil y el modo difícil. En el modo fácil comienza con tres vidas y las gotas caen con menor frecuencia. En el modo difícil comienza con dos vidas y las gotas caen más seguido. Cada gota buena recolectada suma 10 puntos, mientras que cada gota dañina que toca el tarro hace perder una vida.

Narrativa

El juego se desarrolla durante una lluvia en la que caen gotas buenas y dañinas. El jugador controla un tarro y debe moverse para recoger las gotas buenas y esquivar las dañinas. A medida que juega, intenta mantenerse con vida a toda costa.

Elementos

Tarro: es el elemento que controla el jugador para recolectar las gotas. Se mueve horizontalmente por la parte inferior de la pantalla.

Gotas buenas: al recolectarlas, suman 10 puntos.

Gotas dañinas: al tocar el tarro, hacen perder una vida.

Puntaje: muestra los puntos acumulados durante la partida. En la pantalla aparece con el nombre “Gotas totales”.

Vidas: indica cuántas vidas le quedan al jugador. El modo fácil comienza con tres y el modo difícil con dos.

HighScore: muestra el mejor puntaje del modo elegido mientras el juego está abierto. Cada modo tiene su propio récord.

Modos de juego: en el modo fácil aparecen gotas con menor frecuencia y en el modo difícil con mayor frecuencia, además de tener una vida menos. El modo difícil también usa un fondo rojo.

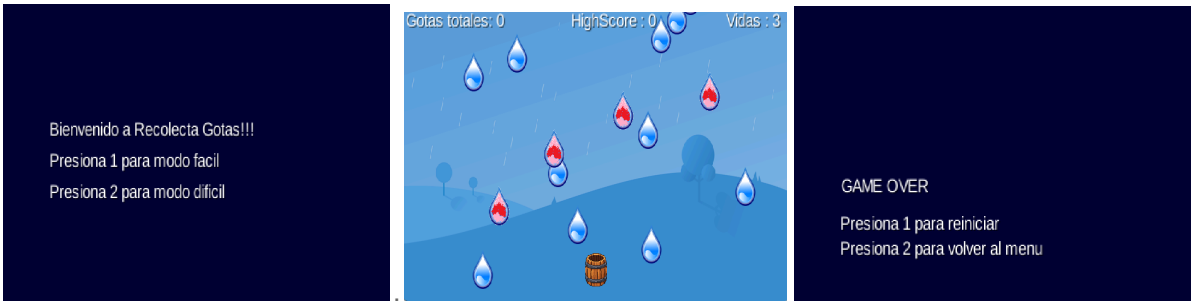
Pantallas: tenemos un menú principal, una pantalla donde se desarrolla la partida y una pantalla de fin del juego.

Sonidos: se escucha lluvia de fondo y efectos al recolectar gotas buenas o recibir daño.

Controles

Para comenzar una partida, el jugador presiona 1 para elegir el modo fácil o 2 para elegir el modo difícil. Durante el juego puede usar las flechas izquierda y derecha para mover el tarro. También puede presionar la tecla 1 para usar las flechas o la tecla 2 para mover el tarro con el mouse. El tarro no puede salir de los bordes de la pantalla.

Al terminar la partida, puede presionar 1 para reiniciar en el mismo modo o 2 para volver al menú y elegir la dificultad de nuevo.



GM-2 Análisis del juego desde la perspectiva de ingeniería de software

Requisito solicitado: Análisis del juego desde la perspectiva de ingeniería de software, especificando el juego base y las modificaciones propuestas.

Modificaciones propuestas

Juego base utilizado

El proyecto utiliza como base **GameLluviaMenu2024**, desarrollado en Java con LibGDX, según el README del repositorio. Es un juego de recolección en el que el jugador controla un tarro y debe capturar gotas beneficiosas mientras evita las dañinas. El proyecto utiliza Java 11 y Gradle. README.

Funcionamiento del juego base

El tarro se desplaza horizontalmente mediante las flechas izquierda y derecha, manteniéndose dentro de los límites de la pantalla. Cada partida comienza con tres vidas y cero puntos.

Las gotas aparecen en posiciones horizontales aleatorias y caen hacia la parte inferior de la pantalla. Al recoger una gota buena, el jugador obtiene diez puntos; al tocar una dañina, pierde una vida. Las gotas que salen por la parte inferior se eliminan sin modificar el puntaje ni las vidas.

Cuando se agotan las vidas, se muestra la pantalla de fin de partida y se permite reiniciar. El juego también dispone de menú inicial, pantalla de pausa, efectos de sonido y música de lluvia. El puntaje máximo se conserva entre partidas durante la ejecución de la aplicación, pero no se guarda en un archivo. Este comportamiento se encuentra en `Tarro`, `Lluvia`, `GameScreen`, `GameLluviaMenu` y las clases de pantallas.

Análisis de la estructura original

En la versión inicial, `Lluvia` representaba las gotas mediante dos colecciones paralelas: `rainDropsPos`, que almacenaba rectángulos con sus posiciones, y `rainDropsType`, que almacenaba números para identificar su tipo.

Esta organización exigía mantener ambas colecciones sincronizadas. Además, `Lluvia` debía comprobar el tipo de cada gota para decidir qué imagen dibujar y qué efecto aplicar al tarro. Esto concentraba varias responsabilidades en una misma clase y dificultaba incorporar nuevos comportamientos. Código inicial de `Lluvia`.

Modificaciones propuestas y avance actual

Se propone reorganizar las gotas utilizando abstracción, herencia y polimorfismo:

- La clase abstracta `Gota` reúne la posición, imagen, área de colisión y movimiento compartidos.
- `GotaBuena` y `GotaMala` heredan de `Gota` y definen su efecto sobre el tarro.
- La interfaz `ComportamientoGota` establece las operaciones de movimiento, dibujo, consulta de colisiones y salida de pantalla, junto con aplicación del efecto.
- `Lluvia` utiliza una colección de `ComportamientoGota`, sustituyendo las dos colecciones paralelas.

Esta modificación ya está integrada en `main`. Actualmente, `Lluvia` llama a `aplicarEfecto(tarro)` y cada tipo de gota ejecuta su comportamiento: sumar puntos o causar daño. La selección del tipo sigue ocurriendo al crear la gota. Cambios incorporados.

Aplicamos encapsulamiento y principios GRASP y SOLID. Partida coordina la actualización del juego, Puntaje controla los récords y Tarro separa la actualización de su estado del dibujo.

Partes que se mantienen y partes que cambian

Se mantiene del juego base	Se modifica o incorpora
Movimiento horizontal del tarro mediante las flechas.	Representación de cada gota mediante un objeto.
Diez puntos por cada gota buena recolectada.	Clase abstracta `Gota`.
Gotas buenas y dañinas.	Clases `GotaBuena` y `GotaMala`.
Caída de gotas y detección de colisiones.	Interfaz `ComportamientoGota`.
Menú, pausa, fin de partida y reinicio.	Colección única de gotas en `Lluvia`.
Imágenes, sonidos y música existentes.	Distribución del comportamiento específico entre las clases de gotas.
	Dos dificultades: facil y dificil, con diferencias en vidas y frecuencia de gotas
	Control mediante mouse o teclado, seleccionable durante la partida

GM-3 Diseño de diagrama UML con clases del Dominio del juego

Requisito solicitado: Diseño de diagrama UML con clases del Dominio del juego, incluyendo clases base LibGDX y propias, y su código en Java.

Clases del dominio y relaciones

Game (LibGDX): Clase base que administra la pantalla activa del juego.

Screen (LibGDX): Interfaz que define el ciclo de vida de una pantalla.

GameLluviaMenu: Hereda de Game, inicia el juego y entrega los recursos para dibujar.

PantallaBase: Clase abstracta que implementa Screen y define el orden común de las pantallas.

MainMenuScreen, GameScreen, PausaScreen y GameOverScreen: heredan de PantallaBase y muestran el menú, la partida, la pausa y el fin del juego.

Partida: contiene el tarro y la lluvia de una partida.

DirectorPartida, PartidaBuilder y PartidaRecolectorBuilder: preparan la partida según el modo fácil o difícil.

Tarro: representa al jugador, con sus vidas, puntos y posición.

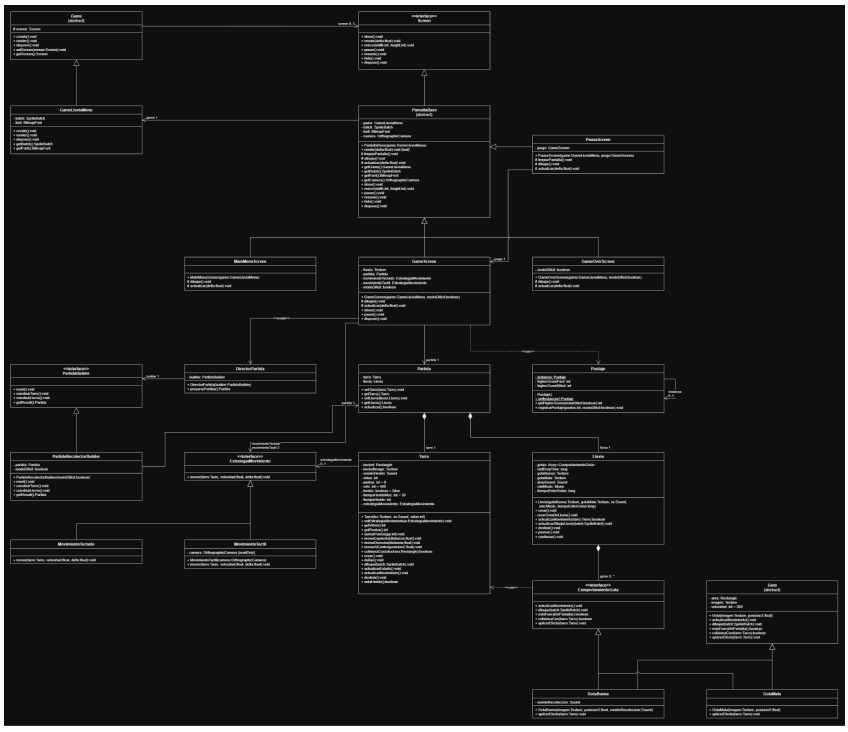
EstrategiaMovimiento, MovimientoTeclado y MovimientoTactil: definen cómo se mueve el tarro, con teclado o con mouse.

Lluvia: crea, mueve y dibuja las gotas.

ComportamientoGota, Gota, GotaBuena y GotaMala: definen las gotas y el efecto de cada tipo sobre el tarro.

Puntaje: guarda el récord de cada modo mientras el juego está abierto.

Diagrama UML



Evidencia de implementación

El diagrama muestra las clases propias del juego y las clases base de LibGDX, con sus atributos, métodos y relaciones. Cada clase está implementada en su propio archivo .java, dentro de core/src/main/java/puppy/code del repositorio.

GM-4 Diseño y codificación de clase abstracta e interfaz

Requisito solicitado: Diseño y codificación de 1 clase abstracta que sea padre de al menos 2 clases y sea utilizada en el programa, y 1 interfaz que sea implementada por al menos 2 clases y utilizada en el programa.

Clase abstracta

La clase abstracta Gota reúne los datos y comportamientos comunes de las gotas: mantiene su posición y su imagen, controla su movimiento y permite dibujarlas y detectar el área de choque. De ella heredan GotaBuena y GotaMala. Cada una define lo que ocurre al tocar el tarro: la primera suma puntos y la segunda quita una vida.

Interfaz

La interfaz ComportamientoGota define las operaciones que se utilizan para manejar los distintos tipos de gotas. GotaBuena y GotaMala la implementan y ambas se pueden actualizar, dibujar, revisar para detectar choques y aplicar su efecto, aunque cada tipo resuelve este último de manera distinta.

Uso en el programa

En Lluvia se crean gotas buenas o malas (que hacen daño) y se guardan como elementos de tipo ComportamientoGota. En el método actualizarMovimiento() de Lluvia se mueve cada gota y se revisa si choca con el tarro. Si hay un choque, se llama a aplicarEfecto(): GotaBuena suma 10 puntos y reproduce el sonido de recolección, mientras que GotaMala llama a dañar() en Tarro y le quita una vida. Para mostrarlas, actualizarDibujoLluvia() llama a dibujar() en cada gota. El movimiento y el dibujo que comparten están implementados en Gota. La evidencia está en: ComportamientoGota.java, Gota.java, GotaBuena.java, GotaMala.java y Lluvia.java.

GM-5 Aplicación de encapsulamiento y principios de orientación a objetos

Requisito solicitado: Aplicación de encapsulamiento y principios de orientación a objetos, incluyendo GRASP y SOLID.

Encapsulamiento

Los atributos de nuestras clases son privados y se consultan o modifican mediante métodos públicos. En Tarro, los datos como vidas, puntos, estado herido y posición se mantienen dentro de la clase. Las estrategias de movimiento utilizan moverIzquierda, moverDerecha y moverAlCentro, evitando modificar directamente su rectángulo interno. Gota también mantiene su área privada y permite consultar si salió de la pantalla o si colisiona con el tarro.

Principios GRASP

Experto en Información:

Puntaje conoce el récord y realiza su comparación mediante registrarPuntaje. Partida contiene al tarro y la lluvia, por lo que su método actualizar coordina la actualización de ambos. Tarro y Gota utilizan sus propios datos para comprobar las colisiones.

Alta Cohesion:

organizamos las operaciones según las responsabilidades de cada clase. Partida coordina la actualización del juego y GameScreen mantiene la selección de controles y los cambios de pantalla. En Tarro se separó actualizarEstado de dibujar, de modo que el tiempo herido se actualiza sin depender de la ejecución del dibujo.

Polimorfismo:

Tarro utiliza la interfaz EstrategiaMovimiento para solicitar el movimiento. Según la estrategia asignada, se ejecuta la implementación de MovimientoTeclado o MovimientoTactil, sin que el tarro tenga que comprobar cuál de las dos está utilizando.

Principios SOLID

Abierto/Cerrado

El movimiento del tarro se puede extender agregando otra clase que implemente EstrategiaMovimiento. Tarro mantiene la misma llamada a mover y las estrategias existentes conservan su código. La creación y selección de una estrategia nueva se configura en GameScreen. La evidencia está en: Tarro.java, Gota.java , Puntaje.java, Partida.java, EstrategiaMovimiento.java, MovimientoTeclado.java y MovimientoTactil.java.

GM-6 Aplicación del patrón de diseño Singleton

Requisito solicitado: Aplicación del patrón de diseño Singleton, describiendo problema, contexto, solución y UML.

Problema

En la versión inicial, el puntaje más alto se guardaba en GameLluviaMenu, la clase que inicia el juego, entrega los recursos para dibujar y cambia entre pantallas. GameScreen tenía que pedirle el récord a esa clase para mostrarlo, y al terminar la partida era la misma pantalla la que comparaba los puntos y lo reemplazaba con setHigherScore. Como ese método era públic, cualquier clase con acceso al juego podía cambiar el récord sin ninguna revisión.

Contexto

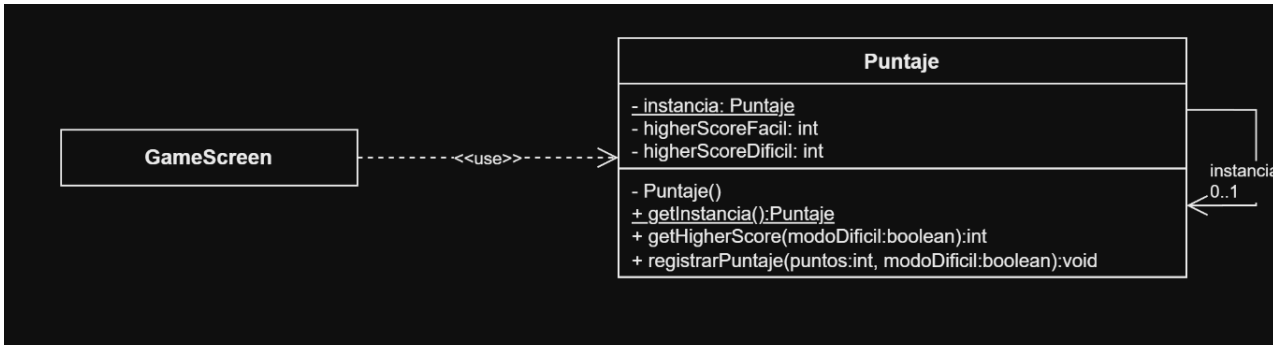
Nuestro juego crea un GameScreen nuevo cada vez que comienza una partida y lo elimina cuando el jugador pierde todas sus vidas. Aun así, el récord tiene que mantenerse entre una partida y otra mientras el juego esté abierto, por lo que no se puede guardar en GameScreen ni en Partida. Queríamos que existiera un solo récord para todo el juego y que la pantalla de partida pudiera consultarlo para mostrarlo y actualizarlo al terminar. Además, el juego guarda un récord distinto para el modo fácil y otro para el modo difícil.

Solución

Creamos la clase Puntaje aplicando el patrón Singleton. Su constructor es privado, así que ninguna otra clase puede crear un objeto Puntaje. La clase guarda su única instancia en el atributo estático instancia, y el método estático getInstancia la crea la primera vez que se pide y la entrega en las llamadas siguientes. De esta forma, todas las partidas usan el mismo objeto y el récord se conserva mientras el juego está abierto.

Sacamos el récord y sus métodos de GameLluviaMenu, que quedó encargándose solo de los recursos y las pantallas. GameScreen usa getInstancia en dos momentos: al dibujar, para mostrar el récord del modo elegido con getHigherScore, y al terminar la partida, para registrar los puntos con registrarPuntaje. Este último método compara los puntos con el récord del modo jugado y solo lo reemplaza si son mayores, por lo que ninguna clase puede modificarlo directamente.

En el patrón, Puntaje corresponde a la clase Singleton, instancia es la referencia a su único objeto y getInstancia es el punto de acceso que utiliza GameScreen. La evidencia está en: Puntaje.java y GameScreen.java



GM-7 Aplicación del patrón de diseño Strategy

Requisito solicitado: Aplicación del patrón de diseño Strategy, describiendo problema, contexto, solución y UML.

Problema

En la versión inicial, Tarro leía directamente las flechas del teclado en actualizar Movimiento. También tenía comentada una alternativa para mouse o toque, la cual dependía de la cámara de la pantalla. Para agregar otra forma de control era necesario modificar la lógica de movimiento de la clase.

Contexto

Nuestro juego permite mover un tarro horizontalmente para recoger gotas. Queríamos utilizar teclado o mouse sin cambiar el objeto que guarda las vidas, los puntos y la posición del jugador. Ambas formas de control realizan la misma operación, pero interpretan la entrada de manera diferente.

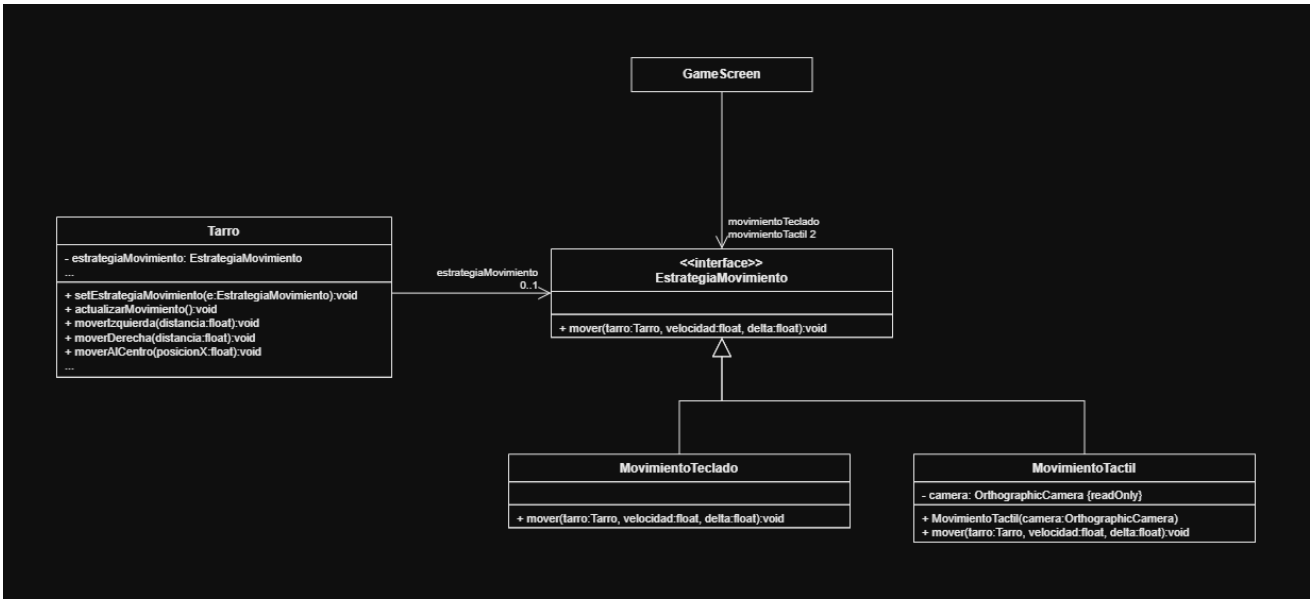
Solución

Creamos la interfaz EstrategiaMovimiento, que define el método mover. MovimientoTeclado implementa este método utilizando las flechas y calcula el desplazamiento según la velocidad y el tiempo transcurrido. MovimientoTactil recibe la cámara, convierte la posición del mouse o toque a coordenadas del juego y coloca el tarro en esa posición.

Tarro guarda una referencia de tipo EstrategiaMovimiento y llama a su método mover desde actualizarMovimiento. Después aplica los límites para que el tarro no salga de la pantalla. Las estrategias utilizan los métodos públicos del tarro, sin acceder directamente a su rectángulo.

GameScreen crea ambas estrategias y asigna inicialmente el teclado. Durante la partida, la tecla 1 selecciona teclado y la tecla 2 selecciona mouse o toque. El cambio reemplaza la estrategia del mismo tarro, conservando sus vidas y puntos.

En el patrón, EstrategiaMovimiento corresponde a Strategy, MovimientoTeclado y MovimientoTactil son las estrategias concretas, y Tarro es el contexto que guarda y utiliza la estrategia. GameScreen se encarga de crearla y seleccionarla. La evidencia está en EstrategiaMovimiento.java, MovimientoTeclado.java, MovimientoTactil.java, Tarro.java y GameScreen.java.



GM-8 Aplicación del patrón de diseño Template Method

Requisito solicitado: Aplicación del patrón de diseño Template Method, describiendo problema, contexto, solución y UML.

Problema

Las pantallas del juego tienen que seguir un orden común para mostrarse: limpiar la pantalla, preparar la cámara, dibujar su contenido y actualizar lo que ocurre. Pero cada pantalla muestra cosas distintas y responde a controles diferentes. Si cada una tuviera que organizar todo el proceso por su cuenta, se repetirían los mismos pasos en varias clases.

Contexto

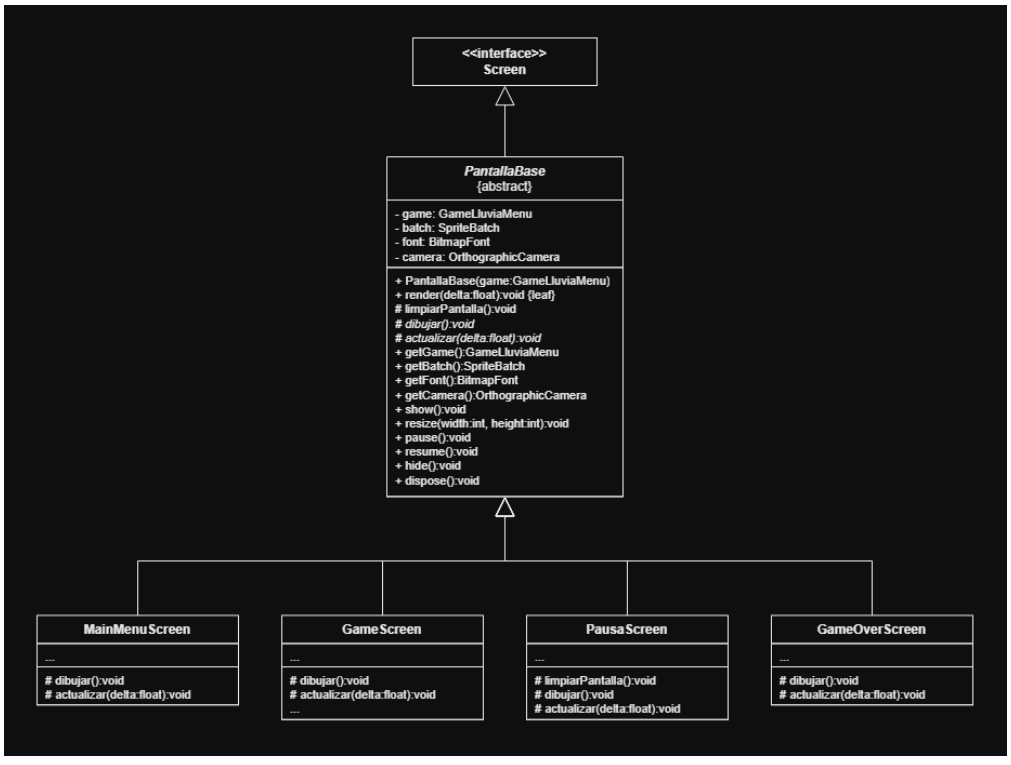
El juego tiene un menú principal, una pantalla de partida, una pantalla de pausa y una pantalla de fin del juego. Cada una se comporta de forma distinta. En el menú se elige el modo fácil o difícil. Durante la partida se muestran el tarro, las gotas, el puntaje y las vidas. La pantalla de pausa permite volver a la partida, mientras que al terminar el juego se puede reiniciar en el mismo modo o volver al menú para elegir otra dificultad.

Solución

Creamos la clase abstracta PantallaBase para dejar en un solo lugar el orden que siguen las pantallas. Su método render primero limpia la pantalla, actualiza la cámara, prepara el dibujo, llama al método dibujar, termina el dibujo y después llama a actualizar. El método render está marcado como final, así que las pantallas que heredan de PantallaBase mantienen ese mismo orden.

Cada pantalla completa las partes que cambian. MainMenuScreen, GameScreen, PausaScreen y GameOverScreen tienen sus propios métodos dibujar y actualizar. Así, cada una muestra su contenido y responde a sus controles sin tener que repetir el proceso completo.

PausaScreen también cambia la forma de limpiar la pantalla para mostrar un color distinto mientras el juego está en pausa. En GameOverScreen, el jugador puede presionar 1 para reiniciar en el mismo modo o 2 para volver al menú. Estos cambios mantienen el mismo proceso general de PantallaBase, aunque cada pantalla tenga un contenido y unas acciones diferentes. La evidencia está en PantallaBase.java, MainMenuScreen.java, GameScreen.java, PausaScreen.java y GameOverScreen.java.



GM-9 Aplicación del patrón de diseño Builder

Requisito solicitado: Aplicación del patrón de diseño Builder, describiendo problema, contexto, solución y UML.

Problema

Para comenzar una partida hay que preparar varios elementos, como el tarro, las gotas, los sonidos y la música. Además, el tarro y la lluvia cambian según la dificultad elegida. Si toda esa preparación se hiciera directamente en la pantalla de juego, esa clase tendría que encargarse de muchas cosas al mismo tiempo.

Contexto

Desde el menú principal, el jugador elige el modo fácil o el modo difícil. Después se abre la pantalla de partida, que prepara los elementos que necesita el juego.

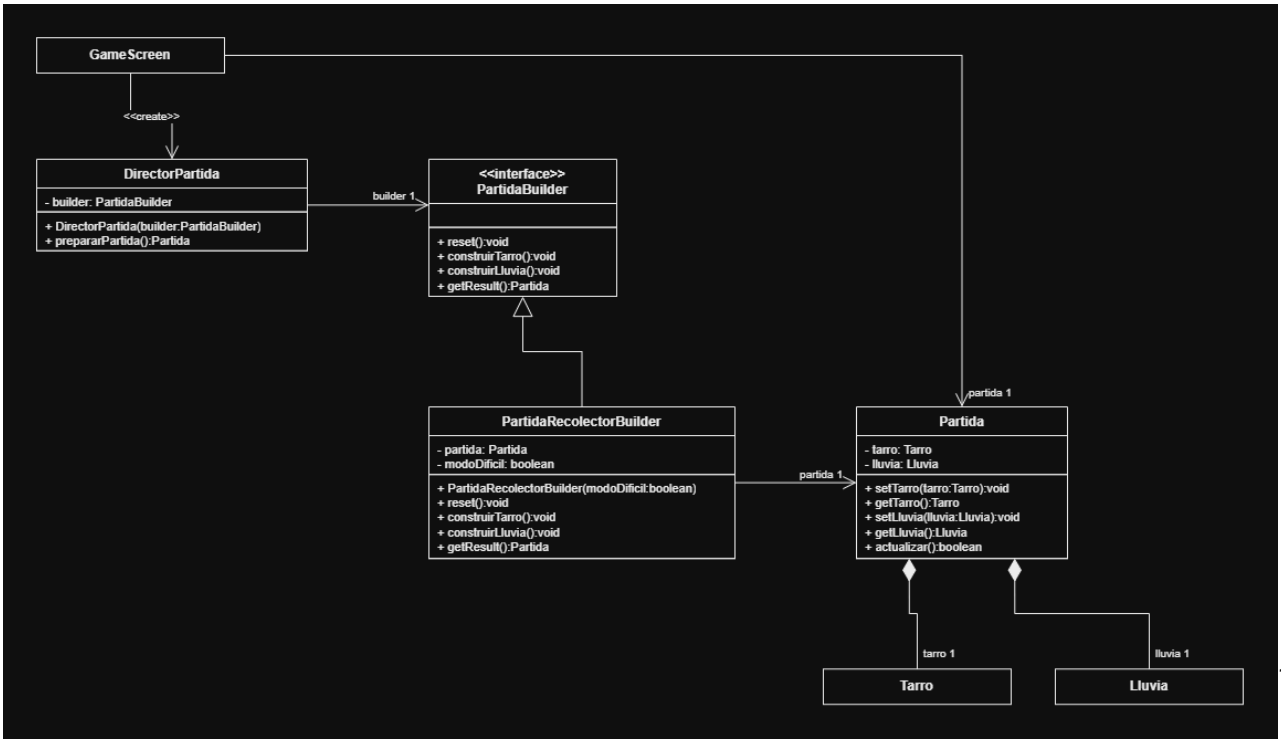
Los dos modos usan un tarro y una lluvia, pero comienzan con distintas vidas y las gotas aparecen con distinta frecuencia. El modo fácil comienza con tres vidas y las gotas aparecen más lento. El modo difícil comienza con dos vidas y las gotas aparecen más seguido.

Solución

Creamos PartidaRecolectorBuilder para preparar los elementos de una partida. El builder crea el tarro y la lluvia, y configura sus valores según el modo elegido. En el modo fácil asigna tres vidas al tarro y hace que las gotas aparezcan más lento. En el modo difícil asigna dos vidas y hace que aparezcan más seguido.

GameScreen crea el builder con el modo elegido y lo entrega a DirectorPartida. El director le pide al builder que prepare el tarro y la lluvia, y después obtiene la partida completa. Esa partida contiene ambos elementos y queda lista.

Lo hicimos así para dejar la preparación de la partida en PartidaRecolectorBuilder, en vez de poner todos esos pasos directamente en GameScreen. El fondo también cambia según el modo, pero se carga desde GameScreen y no lo prepara el builder. La evidencia está en PartidaBuilder.java, PartidaRecolectorBuilder.java, DirectorPartida.java, Partida.java y GameScreen.java.



GM-10 Utilización de GitHub

Requisito solicitado: Utilización de GitHub, realizando al menos 6 commits.

Utilizacion

Utilizamos github para guardar y organizar el trabajo del grupo, donde cada integrante trabaja en su rama remota personal y la rama main mantiene la versión integrada del proyecto.

Las tareas se organizaron mediante issues, repartidos de forma equitativa y lógica entre integrantes, donde en conjunto se integran las partes al proyecto.

La rama main cumple con el requisito mínimo de 6 commits, con un total de 22 commits.

Repositorio

https://github.com/The-Nick-J/GM-Lluvia